

Python Automata Library (PAL) Cheat Sheet

Conventions

import PythonAutomataLibrary as pal · Safe mode is default. Call **pal.FastMode()** before constructing any PAL object. Put substantive model kernels under **@pal.njit(cache=True)**. **dims** gives grid size per axis: **(nx,)** for 1D, **(nx, ny)** for 2D, **(nx, ny, nz)** for 3D; **dims=()** is nonspatial. **x...** means **x**, **x,y**, or **x,y,z** in 1D/2D/3D. **Box(x1,x2,...)** alternates lower and exclusive upper bounds per axis. Negative dimensions enable wrapping; out-of-range coordinates wrap on wrapped axes and are skipped on nonwrapped axes.

Create state

pal.NewGrid(dims, dtype) — Create a typed lattice storing one value per site.
pal.NewAgentGrid(dims, numAgentProps=0, isStackable=False) — Create agents with optional properties; **dims=()** makes a nonspatial grid.
pal.NewPopGrid(dims, capacity=None) — Create integer counts per site; capacity limits total population.
pal.NewPDEgrid(dims) — Create a continuous field for PDE updates.
pal.NewList() — Create a mutable integer list for collecting/reusing query results.
pal.NewMultinomial() — Create a sampler for binomial/multinomial population draws.
Dimensions — 1–3 axes; a negative dimension wraps that axis, e.g. **(-nx, ny)**.

Shared lattice geometry

Properties — **xDim/yDim/zDim, nDims, wrapX/Y/Z**; a negative dimension enables wrapping on that axis.
Tol(x...) — Convert 1D/2D/3D coordinates into one linear site index.
ltoX/Y/Z(i) — Convert a linear site index back to its axis coordinate.
Regions — **Box(x1,x2,...)** selects a rectangular region; each axis has a lower bound and exclusive upper bound.
pal.MooreHood(dim, excludeCenter=False) — Build offsets for all neighboring sites, including diagonals.
pal.VonNeumannHood(dim, excludeCenter=False) — Build axis-aligned offsets, excluding diagonal neighbors.
pal.CircleHood(dim, rad, excludeCenter=False) — Build offsets within a radius; hood constructors return relative offsets, not absolute indices.
grid.Hood(hood, x...) — Apply relative offsets at a coordinate; **excludeCenter=True** omits the center when constructing the hood.

Grid / common indexing

g[i] — Read or write a site by linear index.
g[x...] — Read or write a site using dimensional coordinates.
Slices — Return detached NumPy copies, not live views; editing a slice does not update the grid.
Pattern — **g = pal.NewGrid((40,40), float) · g[10,12] = 1.0 · v = g[10,12]**; supported dtypes include bool, fixed-width integers, and float32/float64.

Draw / output

pal.StartPixWindow(...) — Create a pixel buffer and display window; **headless=True** supports noninteractive rendering.
pix[x,y] = RGB — Set a pixel color in the shared pixel buffer.
win.Update() — Refresh the displayed window after drawing.
win.Save(path, block=True) — Save the current image; blocking waits for completion.
win.Close() — Close the window and release display resources.
StartGif(path,delay=100) — Begin recording an animated GIF with the requested frame delay.
AddGifFrame(block=False) — Append the current rendered frame to the GIF.
StopGif() — Finish the GIF; update the window before capturing frames.
OpenGL — **pal.StartOpenGLWindow(...)**; draw with **Circle, Box, BoxSQ, Line, Borders**; scene controls include **Camera, Background, Clear**.

Lists + randomness

lList methods — **Append(i)** adds an integer; **Clear()** empties the list; **Random()** picks an entry; **Shuffle()** reorders entries; **All()** copies the list; **Iter()** traverses without copying.
pal.Seed(seed) — Seed PAL's shared random stream for reproducible call sequences.
pal.Random() — Draw a uniform random number from PAL's random stream.
pal.RandInt(n) — Draw an integer from 0 through n–1; n must be positive. PAL's stream is separate from NumPy's RNG.
m.Binomial(n,p) — Draw a binomial count with n trials and probability p.
m.Setup(...) / **m.Sample(...)** — Configure a multinomial sampler, then draw counts from it.

AgentGrid

NewAgentSQ(x...) / **MoveSQ(a,x...)** — Create or move an agent using lattice-site coordinates; documented linear-site forms are also supported.
NewAgent(x...) / Move(a,x...) — Create or move an agent using continuous coordinates rather than lattice sites.
Agent state — **grid[a,p]** reads/writes property **p**; **l(a)** is linear site index; **XSQ/YSQ/ZSQ(a)** are lattice coordinates; **X/Y/Z(a)** are continuous coordinates. Check **Alive(a)** before use when agents may be disposed; **Dispose(a)** removes an agent.
GetPop() — Count agents currently in the grid.
AgentsAt(x...) / LastAgent(x...) — Find agents at a site or retrieve the last agent there.
AgentsInRadius(...) — Find agents near a position; wrapped displacement respects periodic boundaries.
counts[x...] — Read site occupancy counts without modifying them.
Iteration — **for a in agents.All()**: ... iterates a detached snapshot and is safe for structural mutation such as **Dispose**.
AgentsInRadius(...) returns nearby agents; wrapped displacements account for periodic boundaries.
DispWrapX/Y/Z(p1,p2) — Compute the shortest displacement along a periodic axis.

PopGrid + PDEgrid

Transactional update — **Add(v, ...)** accumulates pending changes; **Update()** applies them together. **Reset()** clears both current and pending state. Direct **grid[...] = v** changes current state immediately, outside the transaction.
Population — **pop.GetPop()** total; **pop.All()** nonzero site indices. **capacity** limits total population.
PDE setup — **field.SetTimeSpaceStep(dt, dx[,dy,dz])** sets time and spatial steps before updates. Use spacing values that satisfy the selected scheme's stability requirements.
Diffusion — **Diffusion, DiffusionMask, DiffusionField, DiffusionInterfaces, DiffusionADI**; radial 1D: **DiffusionRadialCircle/Sphere**.
Advection — **Advection, AdvectionField, AdvectionInterfaces**.
More detail: **Manual · API Guide · API Reference (Documentation/)**.